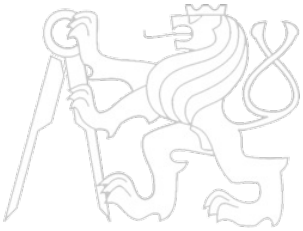
Middleware Architectures 1

Lecture 6: Communication Protocols

doc. Ing. Tomáš Vitvar, Ph.D.
tomas@vitvar.com • @TomasVitvar • https://vitvar.com



Czech Technical University in Prague
Faculty of Information Technologies • Software and Web Engineering • https://vitvar.com/lectures

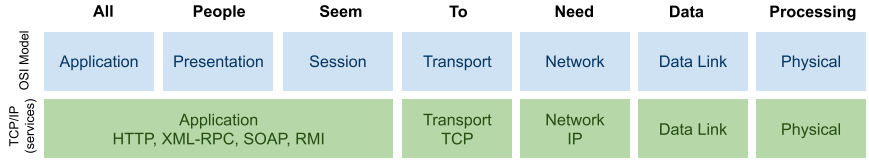
Modified: Sun Nov 23 2025, 20:15:18
Humla v1.0

Overview

- Introduction to Application Protocols
- Hypertext Transfer Protocol (HTTP)
- Security

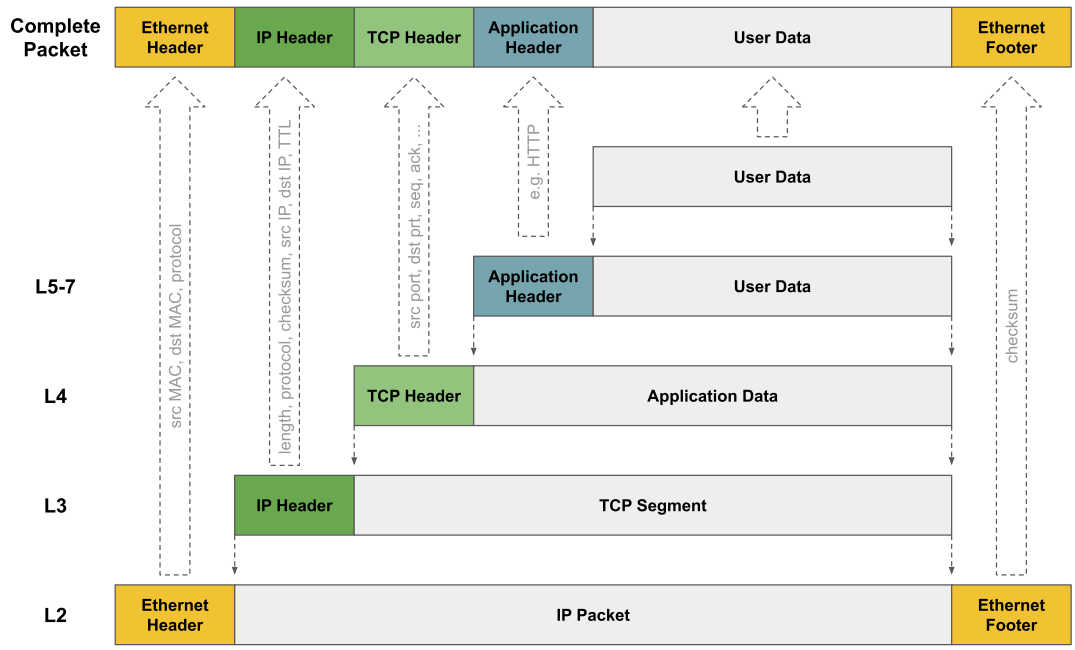
Application Protocols

- Remember this

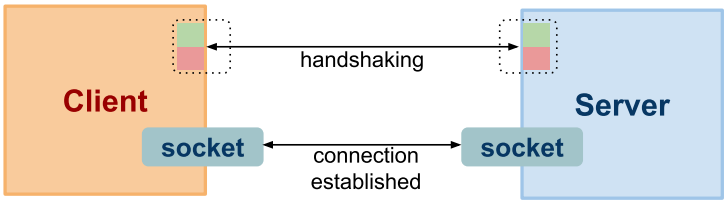


- App protocols mostly on top of the TCP Layer
  - *use TCP socket for communication*
- Major protocols
  - *HTTP – most of the app protocols layered on HTTP*
  - *HTTP/2 – new binary framing*
  - *gRPC – modern RPC protocol on top of HTTP/2*
  - *WebSocket – new protocol part of HTML5*
- Legacy
  - *RMI – Remote Method Invocation*
    - *→ Java-specific; vendor-interoperability problem*
  - *XML-RPC and SOAP – Remote Procedure Call and SOAP*

Anatomy of a Packet



Socket



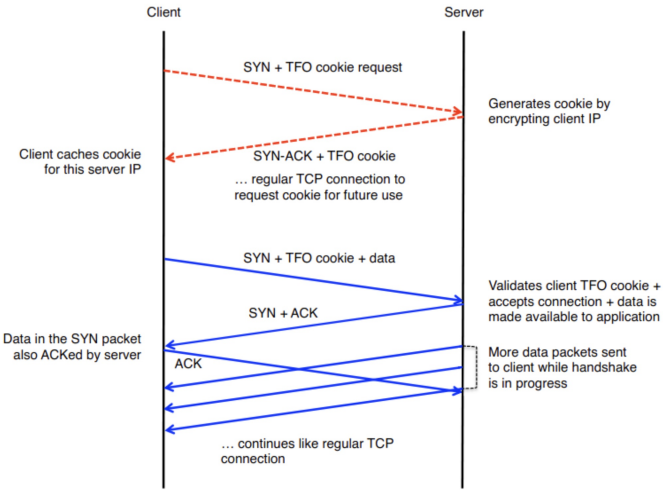
- Handshaking (connection establishment)
  - *The server listens at* `[dst_ip,dsp_port]`
  - *Three-way handshake:*
    - → *the client sends a connection request with TCP flags (SYN, x=rand)*
    - → *the server respons with its own TCP flags (SYN ACK, x+1 y=rand)*
    - → *the client acknowledges the response, can send data along (ACK, y+1 x+1)*
  - *Result is a socket (virtual communication channel) with unique identification:* `socket=[src_ip,src_port;dst_ip,dst_port]`
- Data transfer (resource usage)
  - *Client/server writes/reads data to/from the socket*
  - *TCP features: reliable delivery, correct order of packets, flow control*
- Connection close

New Connection Costs

- Creating a new TCP connection is expensive
  - *It requires to complete a full roundtrip*
  - *It is limited by a network latency, not bandwidth*
- Example
  - *Distance from London to New York is approx. 5500 km*
  - *Communication over a fibre link will take at least 28ms one way*
  - *Three-way handskake will take a minimum of 56ms*
- Connection reuse is critical for any app running over TCP
  - *HTTP Keep-alive*
  - *HTTP pipelining*

TCP Fast Open

- TFO allows to speed up the opening of successfive TCP connections
  - *TCP cookie stored on the client that was established on initial connection*
  - *The client sends the TCP cookie with SYN packet*
  - *The server verifies the TCP cookie and can send the data without final ACK*
  - *Can reduce network transaction latency by 15%*



Overview

- Introduction to Application Protocols
- Hypertext Transfer Protocol (HTTP)
  - *Performance*
  - *Configuration*
  - *HTTP in Kubernetes*
- Security

Hypertext Transfer Protocol – HTTP

- Application protocol, basis of Web architecture
  - *Part of HTTP, URI, and HTML family*
  - *Request-response protocol*
- One socket for single request-response
  - *original specification*
  - *have changed due to performance issues*
    - → *many concurrent requests*
    - → *overhead when establishing same connections*
    - → *HTTP 1.1 offers persistent connection and pipelining*
    - → *Domain sharding*
- HTTP is stateless
  - *Multiple HTTP requests cannot be normally related at the server*
    - → *"problems" with state management*
    - → *REST goes back to the original HTTP idea*

REST Core Principles

- REST architectural style defines constraints
  - *if you follow them, they help you to achieve a good design, interoperability and scalability.*
- Constraints
  - *Client/Server*
  - *Statelessness*
  - *Cacheability*
  - *Layered system*
  - *Uniform interface*
- Guiding principles
  - *Identification of resources*
  - *Representations of resources and self-descriptive messages*
  - *Hypermedia as the engine of application state (HATEOAS)*

HTTP Request and Response

- Request Syntax

```
method uri http-version <crlf>
(header : value <crlf>)*
<crlf>
[ data ]
```

- Response Syntax

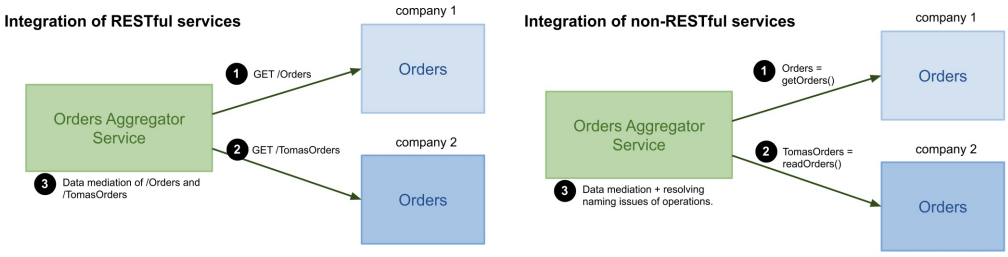
```
http-version response-code [ message ]  <crlf>
(header : value <crlf>)*
<crlf>
[ data ]
```

- Semantics of terms

```
method         = "GET" | "POST" | "DELETE" | "PUT" | "HEAD" | "OPTIONS"
uri            = [ path ] [ ";" params ] [ "?" query ]
http-version   = "HTTP/1.0" | "HTTP/1.1"
response-code  = valid response code
header : value = valid HTTP header and its value
data           = resource state representation (hypertext)
```


Uniform Interface

- Uniform interface = finite set of operations
  - *Resource manipulation*
    - → *CRUD – Create (POST/PUT), Read (GET), Update (PUT/PATCH), Delete (DELETE)*
  - *operations are not domain-specific*
    - → *For example,* `GET /orders` *and not* `getOrders()`
    - → *This reduces complexity when solving interoperability*
- Integration issues examples



Safe and Unsafe Operations

- Safe operations
  - *Do not change the resource state*
  - *Usually "read-only" or "lookup" operation*
  - *Clients can cache the results and refresh the cache freely*
- Unsafe operations
  - *May change the state of the resource*
  - *Transactions such as buy a ticket, post a message*
  - *Unsafe does not mean dangerous!*
- Unsafe interactions and transaction results
  - `POST` *response may include transaction results*
    - → *you buy a ticket and submit a purchase data*
    - → *you get transaction results*
    - → *and you cannot bookmark this..., why?*
  - *Should be referable with a persistent URI*

Idempotence

- Idempotent operation
  - *Invoking a method on the same resource always has the same effect*
  - *Operations* `GET`, `PUT`, `DELETE`
- Non-idempotent operation
  - *Invoking a method on the same resource may have different effects*
  - *Operation* `POST`
- Effect = a state change

Overview

- Introduction to Application Protocols
- Hypertext Transfer Protocol (HTTP)
  - *Performance*
  - *Configuration*
  - *HTTP in Kubernetes*
- Security

Persistent connections

- Persistent HTTP connection = HTTP keepalive
  - *TCP established connection used for multiple requests/responses*
  - *Avoids TCP three-way handshake to be performed on every request*
  - *Reduces latency*
  - *FIFO queuing order on the client (request queuing)*
    - *→ dispatch first request, get response, dispatch next request*
- Example: `GET /html`, `GET /css`
  - *server processing time 40ms and 20ms respectivelly*
- Without HTTP keepalive
  - *three-way handshake 84ms before the data is received on the server*
  - *Response received at 152ms and 132ms respectivelly*
  - *The total time is 284ms*
- HTTP keepalive
  - *One TCP connection for both requests*
  - *In our example this will save one RTT, i.e. 56ms*
  - *The total time will be 228ms*

Persistent connections savings

- Each request needs
  - *Without keepalive, 2 RTT of latency*
  - *With keepalive, the first request needs 2 RTT, a following request needs 1 RTT*
- Savings for `N` requests: `(N-1) x RTT`
- Average value of `N` is 90 requests for a Web app
  - *Measured by HTTP Archive (http://httparchive.org) as of 2013*
  - *Average Web application is composed of 90 requests fetched from 15 hosts*
    - → *HTML: 10 reuqests*
    - → *Images: 55 requests*
    - → *Javascript: 15 requests*
    - → *CSS: 5 requests*
    - → *Other: 5 requests*

HTTP pipelining

- Important optimization – response queuing
  - *Allows to relecote FIFO queue from the client to the server*
- Requests are pipelined one after another
  - *This allows the server to process requests immediatelly one after another*
  - *This saves one request and response propagation latency*
  - *In our example, the total time will be 172ms*
- Parallel processing of requests
  - *In our example this saves another 20ms of latency*
  - ***Head of line blocking***
    - → *Slower response (css with processing time 20ms) must be buffered until the first response is generated and sent (no interleaving of responses)*
- Issues
  - *A single slow response blocks all requests behind it*
  - *Buffered (large or many) responses may exhaust server resources*
  - *A failed response may terminate TCP connection*
    - → *A client must request all sub-sequent resources again (dupplicate processing)*
  - *Some intermediaries may not support pipelining and abort connection*
- HTTP pipelining support today is limited

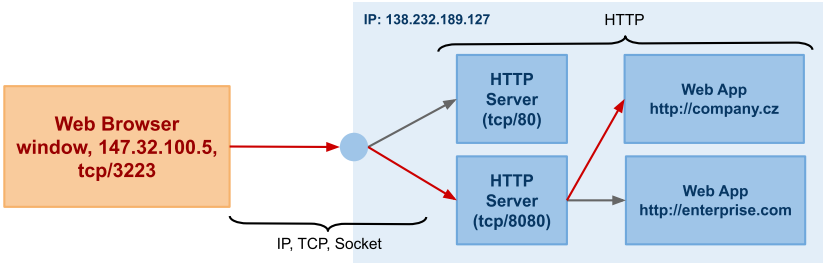
Multiple TCP connections

- Using only one TCP connection is slow
  - *Client must queue HTTP requests and process one after another*
- Multiple TCP connections work in parallel
- **There are 6 connections per host**
  - *The client can dispatch up to 6 requests in parallel*
  - *The server can process up to 6 requests in parallel*
  - *This is a trade-off between higher request paralellism and the client and server overhead*
- The maximum number of connections prevents from DoS attacks
  - *The client could exhaust server resources*
- Domain sharding
  - *The connection limit as per host (origin)*
  - *There can be multiple origins used in a page*
    - → *Each origin has 6 maximum connection limit*
  - *A domain can be sharded*
    - → `www.example.com` → `shard1.example.com`, `shard2.example.com`
    - → *Each shard can resolve to the same IP or different IP, it does not matter*
  - *How many shards?*

Overview

- Introduction to Application Protocols
- Hypertext Transfer Protocol (HTTP)
  - *Performance*
  - *Configuration*
  - *HTTP in Kubernetes*
- Security

Serving HTTP Request



- Serving HTTP request
  1. *User enters URL* `http://shard1.example.com/orders` *to the browser*
  2. *DNS resolution: browser gets an IP address for* `shard1.example.com`
  3. *Three-way handshake: browser and Web Server creates a socket*
  4. *Browser sends ACK and HTTP request:*

```
1 GET /orders HTTP/1.1
2 Host: shard1.example.com
```

  5. *Web server passes the request to the web application* `shard1.example.com` *which serves* `GET orders` *and that writes a response back to the socket.*

Virtual Host

- Virtual host
  - *Configuration of a named virtual host in a Web server*
  - *Web server uses host request header to distinguish among multiple virtual hosts on a single physical host.*
- Apache virtual host configuration
  - *Two virtual hosts in a single Web server*

```
1  # all IP addresses will be used for named virtual hosts
2  NameVirtualHost *:80
3
4  <VirtualHost *:80>
5          ServerName www.example.com
6          ServerAlias shard1.example.com shard2.example.com
7          ServerAdmin admin@example.com
8          DocumentRoot /var/www/apache/example.com
9  </VirtualHost>
10
11 <VirtualHost *:80>
12         ServerName company.cz
13         ServerAdmin admin@firm.cz
14         DocumentRoot /var/www/apache/company.cz
15 </VirtualHost>
```

Overview

- Introduction to Application Protocols
- Hypertext Transfer Protocol (HTTP)
  - *Performance*
  - *Configuration*
  - *HTTP in Kubernetes*
- Security

HTTP in Kubernetes

- Kubernetes does not provide built-in Layer 7 (HTTP) routing
- Services expose Pods at TCP/UDP level
  - `ClusterIP`: *internal-only*
  - `NodePort`: *accessible via* `NodeIP:NodePort`
  - `LoadBalancer`: *cloud-provider load balancer*
- For advanced routing (URLs, hostnames, TLS)
  - *Kubernetes uses the Ingress API*
- Ingress works at the HTTP layer (L7) and provides:
  - *Path-based routing*
  - *Host-based routing*
  - *TLS termination*
  - *Centralized access point*

Ingress Resource

- An Ingress is a set of HTTP routing rules
- It defines how external HTTP/S traffic reaches cluster Services
- An Ingress needs an Ingress Controller
  - *NGINX Ingress Controller*
  - *HAProxy Ingress*
  - *Traefik*
  - *Contour*
- Controller watches Ingress objects and configures
  - *reverse proxy*
  - *load balancer*
  - *dynamic routing tables*

How Ingress Works

- Client sends HTTP request to public IP
  - *LoadBalancer or NodePort*
- Ingress Controller receives the request
  - *Runs as a Deployment inside the cluster*
  - *Listens on ports 80/443*
  - *Uses reverse proxy (NGINX/HAProxy/etc.)*
- Ingress rules decide where to send the request
  - `Host` *matching* → `example.com`
  - `Path` *matching* → `/api`, `/app`
- The controller forwards the request to the Service
  - *The Service then forwards to Pods*
- Ingress Controller handles
  - *TLS termination*
  - *HTTP routing*
  - *Load balancing across Pods*

Ingress Example

- Example Ingress routing traffic to the `busybox-tcp-service`

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: example-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - host: busybox.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: busybox-tcp-service
            port:
              number: 8080
```

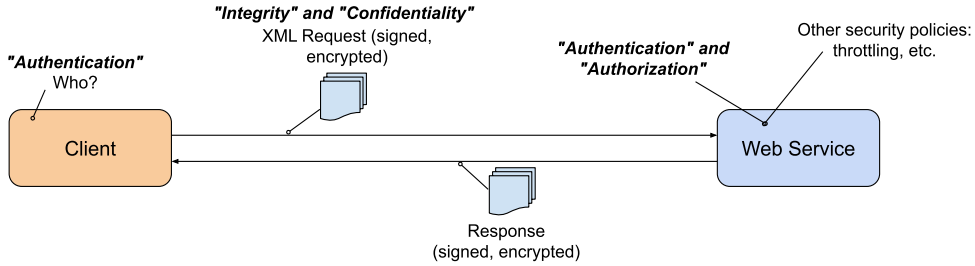
- Requests to `http://busybox.example.com/` are routed to the Service
- Ingress Controller performs HTTP reverse proxying and load balancing across Pods

Overview

- Introduction to Application Protocols
- Hypertext Transfer Protocol (HTTP)
- Security

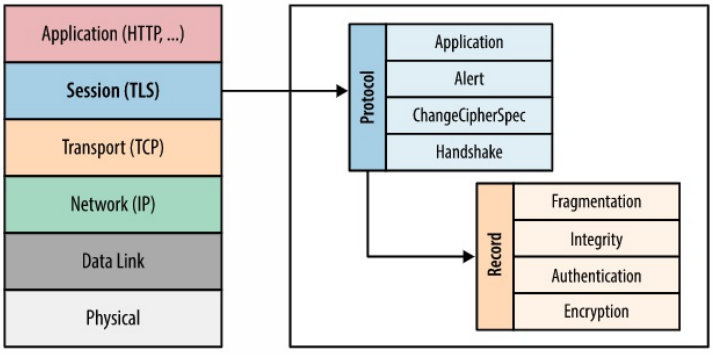
Web Service Security Concepts

- Securing the client-server communcation
  - *Message-level security*
  - *Transport-level security*
- Ensure
  - *Authentication – verify a client's identity*
  - *Authorizaton – rights to access resources*
  - *Message Confidentiality – keep message content secret*
  - *Message Integrity – message content does not change during transmission*
  - *Non-repudiation – proof of integrity and origin of data*



Transport Level Security

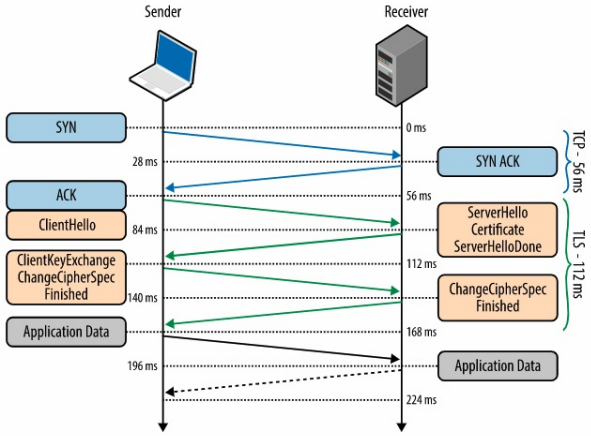
- SSL and TLS
  - *SSL and TLS is used interchangeably*
  - *SSL 3.0 developed by Netscape*
  - *IETF standardization of SSL 3.0 is TLS 1.0*
    → TLS 1.0 is upgrade of SSL 3.0
  - *Due to security flaws in TLS 1.0, TLS 1.1 and TLS 1.2 were created*
- TLS layer



TLS Services

- Encryption
  - *Peers must agree on ciphersuite and keys*
  - *This is achieved by **TLS handshake***
- Authentication
  - *Peers can authenticate their identity*
    - *→ The client can verify that the server is who it is claimed to be*
    - *→ Achieved by "Chain of Trust and Certificate Authoritites"*
    - *→ The server can also verify the client*
- Integrity
  - *TLS provives message framing mechanism*
  - *Every message is signed with Message Authentication Code (MAC)*
    - *→ MAC hashes data in a message and combines the resulting hash with a key (negotiated during the TLS handshake)*
    - *→ The result is a message authentication code sent with the message*

TLS Handshake Protocol



- TLS Handshake

  56 ms: ClientHello, TLS protocol version, list of ciphersuites, TLS options
  84 ms: ServerHello, TLS protocol version, ciphersuite, certificate
  112 ms: RSA or Diffie-Hellman key exchange
  140 ms: Message integrity checks, sends encrypted "Finished" message
  168 ms: Decrypts the message, app data can be sent

Key Exchange

- RSA key exhange(Rivest–Shamir–Adleman)
  - *The client generates a symetric key*
  - *The client encrypts the key with the server's public key*
  - *The client sends the encrpyed key to the server*
  - *The server uses its private key to decrypt the symetric key*
- RSA critical weekness
  - *The same public-private key pair is used to:*
    - → *authenticate the server (the server's private key is used to sign and verify the handshake)*
    - → *encrypt the symetric key*
  - *When an attacker gets hold of the server private key*
    - → *It can decrypt the entire session*
- Diffie-Hellman key exhange
  - *Client and server can negoriate shared secret without its explicit communication*
    - → *Attacker cannot get the key*
  - *Reduction of risk of compromising of the past communications*
    - → *New key can be generated as part of every key exchange*
    - → *Old keys can be discarded*

TLS and Proxy Servers

- TLS Offloading
  - *Inbound TLS connection, plain outbound connection*
  - *Proxy can inspect messages*
- TLS Bridging
  - *Inbound TLS connection, new outbound TLS connection*
  - *Proxy can inspect messages*
- End-to-End TLS (TLS pass-through)
  - *TLS connection is passed-through the proxy*
  - *Proxy cannot inspect messages*
- Load balancer
  - *Can use TLS offloading or TLS bridging*
  - *Can use TLS pass-through with help of Server Name Indication (SNI)*